COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S3
DATE PERFORMED:	09/08/2026	**DATE SUBMITTED:**	09/09/2026
NAME:	Payas, John Paul S.	**INSTRUCTOR:**	Sir. Jimlord Quejado

ACTIVITY NO. 7 **SORTING ALGORITHMS: BUBBLE, SELECTION, AND INSERTION SORT**

1. PROCEDURE OUTPUT

Code + Console Screenshot

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sorting.h"

const int maxSize = 100;

void displayArr(int arr[]);

int main(){

    int unsortedArr[maxSize];

    srand(time(0));

    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()%100;
    }

    std::cout<<"Original Array: \n";
    displayArr(unsortedArr);

    std::cout<<"\nBubbleSort: \n";
    bubbleSort(unsortedArr, maxSize);
    displayArr(unsortedArr);

    std::cout<<"\nSelectionSort: \n";
    selectionSort(unsortedArr, maxSize);
    displayArr(unsortedArr);

    std::cout<<"\nInsertionSort: \n";
    insertionSort(unsortedArr, maxSize);
    displayArr(unsortedArr);

}

void displayArr(int arr[]){

    for(int i = 0; i < maxSize; i++){
        std::cout << arr[i] << " ";
    }

    std::cout << std::endl;
}
```

Observations	The program generated 100 random values from 0 to 99. The values were initially unsorted and were used as input for the three sorting algorithms.

Table 7-1. Array of Values for Sort Algorithm Testing

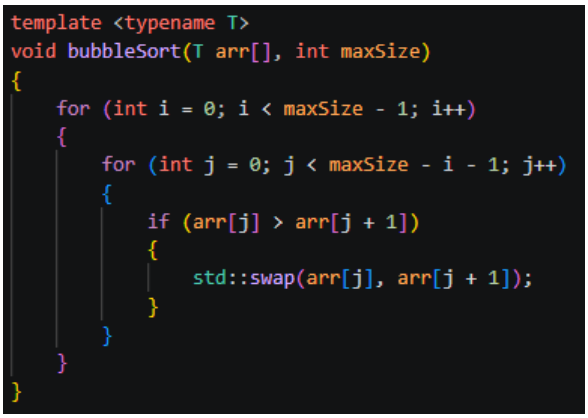
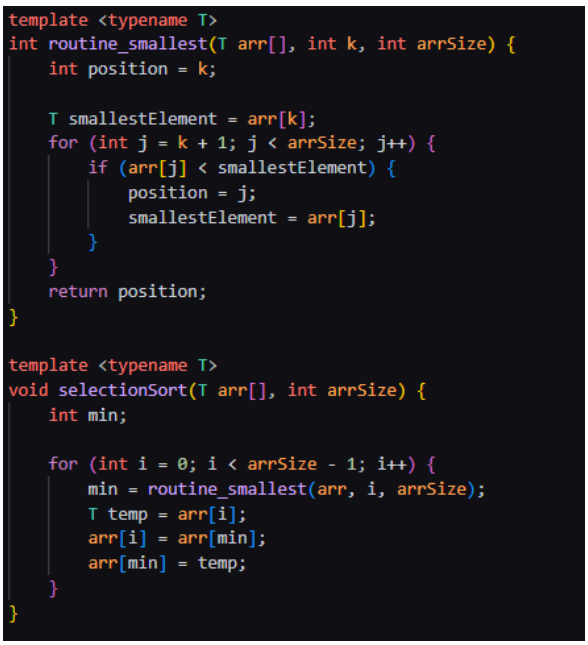
Code + Console Screenshot	
Observations	Bubble Sort successfully arranged all 100 values in ascending order by repeatedly comparing and swapping adjacent elements. Its time complexity is $O(n^2)$ in the average and worst cases.

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	

Observations	Selection Sort successfully arranged the 100 values in ascending order by finding the smallest element and placing it in its correct position. Its time complexity is $O(n^2)$ in the best, average, and worst cases.

Table 7-3. Selection Sort Algorithm

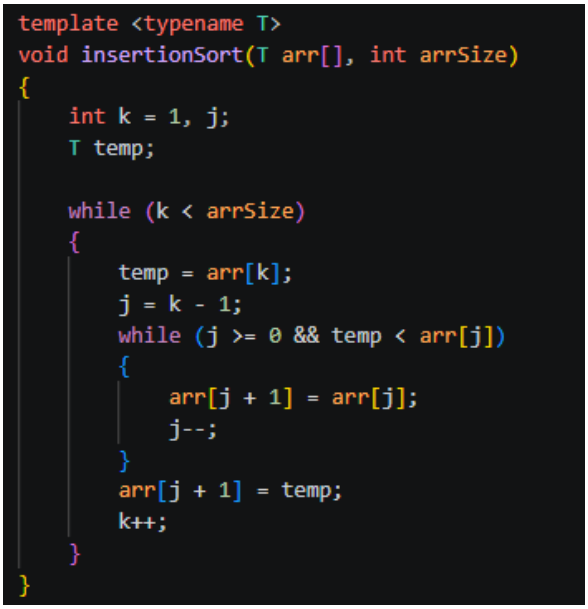
Code + Console Screenshot	
Observations	Insertion Sort successfully arranged all 100 values in ascending order by inserting each element into its proper position. Its time complexity is $O(n^2)$ in the average and worst cases, while its best case is $O(n)$ when the array is already sorted.

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Code:

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sorting.h"

const int maxSize = 100;

void displayArr(int arr[]);

int main()
{
    int votes[maxSize];

    srand(time(0));

    for(int i = 0; i < maxSize; i++)
    {
        votes[i] = rand() % 5 + 1;
    }

    std::cout << "Original Votes:\n";
    displayArr(votes);

    selectionSort(votes, maxSize);

    std::cout << "\nSorted Votes:\n";
    displayArr(votes);

    int vote1 = 0;
    int vote2 = 0;
    int vote3 = 0;
    int vote4 = 0;
    int vote5 = 0;

    for(int i = 0; i < maxSize; i++)
    {
        if(votes[i] == 1)
            vote1++;
        else if(votes[i] == 2)
            vote2++;
        else if(votes[i] == 3)
            vote3++;
        else if(votes[i] == 4)
            vote4++;
        else if(votes[i] == 5)
            vote5++;
```

```
}

std::cout << "\nManual Count:\n";
std::cout << "Vote 1: " << vote1 << std::endl;
std::cout << "Vote 2: " << vote2 << std::endl;
std::cout << "Vote 3: " << vote3 << std::endl;
std::cout << "Vote 4: " << vote4 << std::endl;
std::cout << "Vote 5: " << vote5 << std::endl;

int count[6] = {0};

for(int i = 0; i < maxSize; i++)
{
    count[votes[i]]++;
}

std::cout << "\nCandidate Voting Results:\n";

std::cout << "Candidate 1 - Bo Dalton Capistrano: "
          << count[1] << " votes\n";

std::cout << "Candidate 2 - Cornelius Raymon Agustín: "
          << count[2] << " votes\n";

std::cout << "Candidate 3 - Deja Jayla Bañaga: "
          << count[3] << " votes\n";

std::cout << "Candidate 4 - Lalla Brielle Yabut: "
          << count[4] << " votes\n";

std::cout << "Candidate 5 - Franklin Relano Castro: "
          << count[5] << " votes\n";


int winner = 1;

for(int i = 2; i <= 5; i++)
{
    if(count[i] > count[winner])
    {
        winner = i;
    }
}
```

```
    std::cout << "\nWinner: ";

    if(winner == 1)
        std::cout << "Bo Dalton Capistrano";
    else if(winner == 2)
        std::cout << "Cornelius Raymon Agustín";
    else if(winner == 3)
        std::cout << "Deja Jayla Bañaga";
    else if(winner == 4)
        std::cout << "Lalla Brielle Yabut";
    else
        std::cout << "Franklin Relano Castro";

    std::cout << std::endl;

    return 0;
}

void displayArr(int arr[])
{
    for(int i = 0; i < maxSize; i++)
    {
        std::cout << arr[i] << " ";

        if((i + 1) % 10 == 0)
            std::cout << std::endl;
    }
}
```

Pseudo Code:

```
START

Generate 100 random votes from 1 to 5

Display the original votes

Sort the votes using Selection Sort

Display the sorted votes

Set Vote1, Vote2, Vote3, Vote4, and Vote5 to 0

FOR each vote
    IF vote is 1
        Add 1 to Vote1
    ELSE IF vote is 2
        Add 1 to Vote2
    ELSE IF vote is 3
        Add 1 to Vote3
    ELSE IF vote is 4
        Add 1 to Vote4
    ELSE
        Add 1 to Vote5
END FOR

Display the vote counts

Compare all vote counts

Display the candidate with the highest votes

END
```

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
Original Votes: 1 2 1 4 4 1 3 4 2 5 4 1 1 4 3 3 3 3 1 1 5 2 2 5 3 5 1 5 4 2 2 2 2 4 2 5 1 1 5 3 1 4 3 2 3 5 3 3 5 3 4 3 1 2 3 2 4 4 4 4 1 4 3 3 3 4 5 5 5 5 2 4 3 5 5 5 1 1 3 5 2 2 5 5 2 4 5 5 4 1 2 5 5 5 1 1 5 1 4 4 Sorted Votes: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5	Manual Count: Vote 1: 19 Vote 2: 17 Vote 3: 19 Vote 4: 20 Vote 5: 25	Candidate Voting Results: Candidate 1 - Bo Dalton Capistrano: 19 votes Candidate 2 - Cornelius Raymon Agustín: 17 votes Candidate 3 - Deja Jayla Bañaga: 19 votes Candidate 4 - Lalla Brielle Yabut: 20 votes Candidate 5 - Franklin Relano Castro: 25 votes Winner: Franklin Relano Castro

Question: Was your developed vote counting algorithm effective? Why or why not?

Yes, the algorithm was effective because it correctly counted the votes for each candidate and identified the candidate with the most votes. The results could also be compared with the manual count to verify the accuracy of the algorithm.

Summary of Lessons Learned

I learned how Bubble Sort, Selection Sort, and Insertion Sort work.. I learned that each algorithm uses a different method to arrange elements in ascending order. I also learned how sorting can be combined with counting techniques to solve practical problems.

Analysis of the Procedure

The procedure helped me understand how sorting algorithms work by allowing me to implement and test them using an array of 100 values. The output showed that the algorithms can successfully arrange unsorted values into sorted order when implemented correctly.

Analysis of the Supplementary Activity

The supplementary activity showed how sorting can be applied to vote counting. By sorting the votes and counting each value from 1 to 5, the program was able to determine the number of votes received by every candidate and identify the winner.

Concluding Statement / Feedback

I was able to understand and implement the basic sorting algorithms. However, I still need to improve my understanding of loops, array indexing, and the differences between each sorting algorithm. I can improve by practicing more sorting problems and testing my code with different sets of data.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chat gpt**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **It just fix my bad grammar but it was my idea**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.



I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.